

GAMEFORGE: OPEN-ENDED GAME DEVELOPMENT WITH INDEPENDENT ARTIFACT VERIFICATION

Chen Huang

Singapore University of Technology and Design

ABSTRACT

Generating a game requires more than producing plausible source code: the delivered project must survive engine import, execute its logic, and support observable interaction. We present GameForge, a game-development harness that separates open-ended implementation from independent artifact verification. A coding model receives a complete disposable project and chooses its own tools, engine commands, and development strategy. After the solver exits, a separate, engine-native evaluator assesses the final artifact and records the evidence supporting its verdict. A common execution contract connects model backends, authority profiles, and engine adapters, with implementations for Unity and Godot. On 332 locally qualified GameDevBench tasks, a frozen GameForge configuration solves 213 tasks (64.16%), compared with 184 (55.42%) for a local upstream condition, at 52.8% more tokens. On all 140 GameCraft-Bench tasks, the unified harness produces 140 successful builds and completes 732 interaction replays; its disclosed subscription-judge approximation scores 63.15%. Two authored open-ended suites additionally yield 20/20 Godot import/runtime passes and 20/20 Unity build/launch passes. These results support open-ended, engine-native delivery across two engines, while exposing the distinction between executable artifacts, verified interactions, and overall game quality. We release the harness, evaluation protocols, and selected gameplay evidence to make these distinctions inspectable.

1 INTRODUCTION

Language-model agents increasingly operate on complete software repositories, using execution feedback to turn natural-language requests into code changes (Jimenez et al., 2024; Yang et al., 2024; Wang et al., 2025). Game development extends this challenge to interactive artifacts whose behavior depends jointly on scripts, scenes, assets, input mappings, and engine state. Syntactically valid code may belong to a project that fails to import; a successful build may launch without implementing the requested interaction. Even an attractive rendered scene provides limited evidence about progression, responsiveness, or the consistency of gameplay. Reliable assessment therefore requires observing the delivered project at several stages of engine execution.

Recent benchmarks make these demands measurable. GameDevBench evaluates modifications to existing Godot projects, including visual and behavioral requirements, while GameCraft-Bench evaluates game creation through engine execution, interaction traces, and requirement-level visual judgments (Chi et al., 2026; Luo et al., 2026). The two settings require different development strategies: a localized scene edit and the construction of a complete tower-defense game need not follow the same sequence of tools. A reusable harness must accommodate such variation while maintaining a clear basis for judging the resulting artifact. This motivates separating the solver’s development process from the procedure that determines whether its delivered project satisfies the selected evaluation criteria.

We present GameForge,¹ an engine-native game-development harness organized around *freedom-verification separation*. During implementation, the model chooses how to inspect the project, write

¹Code and evaluation materials: <https://github.com/AlbusChen/GameForge-Harness>. Project page and demonstrations: <https://albuschen.github.io/GameForge-Harness/>.

helpers, invoke the engine, examine images and logs, and revise its approach. The harness provides a complete working project and an execution budget without prescribing a task router, tool sequence, or repair loop. After the solver session ends, a separately executed evaluator applies the selected protocol to the final artifact. Depending on the task, its evidence includes engine tests, builds, runtime observations, input replay, and visual assessment. This division gives the solver flexibility during development and grounds the final verdict in observations of the delivered project.

Our contributions are:

- An artifact-centered execution contract that defines a model-led development session, a complete project handoff, and a subsequent evaluation with explicit supporting evidence.
- A common execution framework for Unity and Godot that separates model invocation, execution authority, and engine-specific operations, supporting both project modification and open-ended creation.
- An empirical study across two external benchmarks and two authored creation suites, reporting task outcomes alongside resource use, interaction evidence, and the scope of each quality measure.

In the same-task GameDevBench comparison, GameForge solves 213 of 332 locally qualified tasks, compared with 184 for the local upstream condition: a gain of 29 tasks, or 8.73 percentage points, with increased resource use. The creation studies extend the evaluation to complete Godot games and open-ended delivery in both engines. Across these settings, we distinguish successful engine execution from demonstrated interaction and rubric-based quality. The resulting evidence characterizes the capabilities and remaining limitations of an open solver interface for game development.

2 RELATED WORK

Software agents and execution interfaces. SWE-bench evaluates repository-level changes against executable tests (Jimenez et al., 2024). SWE-agent studies how the agent-computer interface affects software-engineering performance, while OpenHands provides a platform for agents that interact with code, terminals, and browsers (Yang et al., 2024; Wang et al., 2025). Agentless demonstrates a structured alternative based on localization, repair, and validation (Xia et al., 2024). ReAct and Reflexion establish complementary patterns for interleaving reasoning with actions and incorporating feedback into subsequent attempts (Yao et al., 2023; Shinn et al., 2023). These works motivate treating the execution interface as part of an agent system. GameForge extends this perspective to engine-based artifacts: it leaves development strategy to the solver while specifying the project handoff, engine integration, and evidence used for the final delivery decision.

Systems for game creation. OpenGame organizes web-game generation into a structured workflow with reusable project templates and debugging skills (Jiang et al., 2026). UniGen coordinates planning, C# generation, automated scene construction, and debugging to produce Unity projects (Yang et al., 2025). Both address the integration required to turn generated components into executable games. GameForge investigates a complementary design point: a common solver-session contract across Unity and Godot, with engine-specific execution adapters and post-session evaluation. Its empirical question is whether this comparatively open execution interface supports project modification and game creation under explicit delivery protocols. Iterative editing and testing remain available within the solver’s own session.

Game-development benchmarks. GameDevBench evaluates tutorial-derived Godot development tasks using executable tests; GameCraft-Bench evaluates complete games through replayed demonstrations and rubric-guided multimodal judgments (Chi et al., 2026; Luo et al., 2026). They provide the principal external task distributions and evaluation procedures used here. A2Z GameSpec-Bench further studies faithfulness to long-form game design documents (Lee et al., 2026). These benchmarks expose different aspects of implementation quality, motivating explicit reporting of both the artifact contract and the observed evidence. Learned judging also introduces evaluation choices and biases beyond deterministic test execution (Zheng et al., 2023). Accordingly, our experiments distinguish task-test outcomes, engine gates, replay evidence, and judge-dependent quality scores.

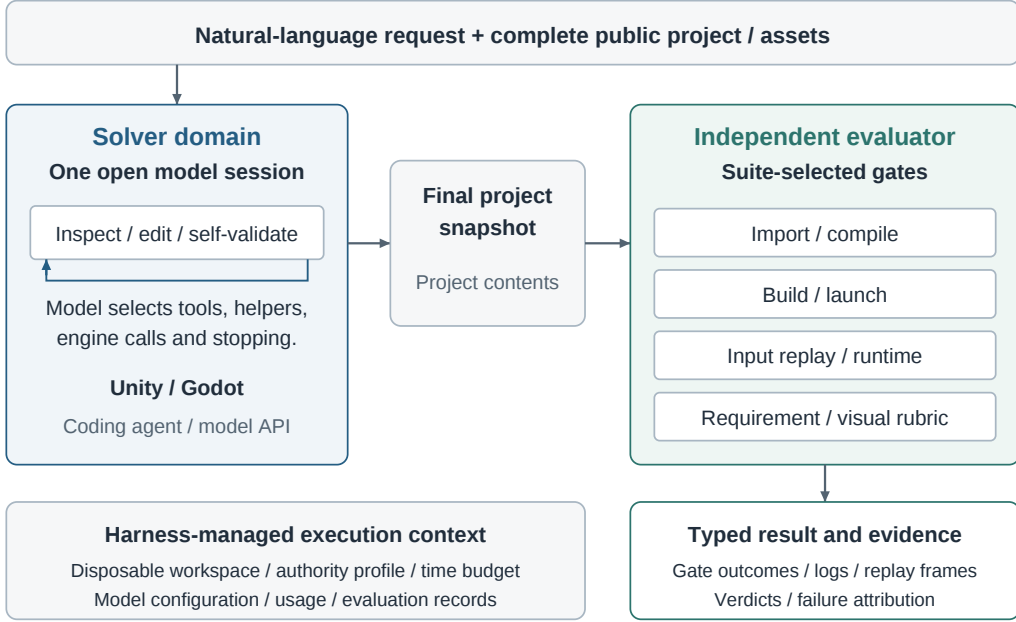


Figure 1: GameForge separates a flexible development session from the final delivery decision. The solver can perform its own validation and revision. The evaluator begins after solver termination and applies the selected suite’s gates. Builds, replays, and visual review are protocol-dependent; they are not all mandatory in every harness invocation.

Generative interactive worlds. Genie and GameNGen learn action-conditioned visual dynamics that support interaction with generated environments (Bruce et al., 2024; Valevski et al., 2025). GameForge targets a different output representation: editable projects executed by existing engines. Scenes, scripts, assets, and input configuration remain inspectable and rebuildable, so evaluation centers on the behavior of the delivered project and the evidence collected from its execution.

3 THE GAMEFORGE HARNESS

3.1 TASK AND ARTIFACT CONTRACT

A task is specified by a natural-language goal q , an initial project P_0 , an engine configuration e , an execution budget b , and an evaluation protocol $\mathcal{E}$. For project modification, P_0 is an existing game; for creation, it is an empty or minimal engine project. The harness materializes a disposable workspace W_0 from P_0 . The solver produces a candidate final project P^* through a sequence of observations and actions:

$$a_t \sim \pi_\theta(\cdot \mid q, o_{\leq t}, a_{< t}), \quad (W_{t+1}, o_{t+1}) = \mathcal{T}_{e,\alpha}(W_t, a_t), \quad (1)$$

where π_θ is the chosen coding model and α specifies execution authority. Actions can include file edits, shell commands, and engine invocations allowed by that profile. The solver’s tool sequence is not specified by the harness.

After termination, the selected evaluator assesses the delivered artifact:

$$(v, z) = \mathcal{E}_e(q, P^*; \rho), \quad r = (\text{id}, e, \alpha, b, u, \tau, v, z), \quad (2)$$

where ρ denotes evaluator configuration, v is the verdict, z is its evidence, u is measured usage, and τ records timing. The record r associates the verdict with the artifact and experimental conditions. A hard-gate pass is defined by the checks chosen by $\mathcal{E}$.

Table 1: Execution authority is configured independently of the model backend. Only native-open and its experimental predecessor underlie the headline results.

Profile	Execution policy	Boundary and intended use
Native-open	Native tools and shell with broad capability	Trusted development environment; no strong host isolation claim.
Supervised-native	Brokered access to pinned engine processes	Engine lifecycle supervision; not a complete hostile-code sandbox.
Strong-isolated	User-supplied VM, container, or remote runner	External provider must supply the isolation boundary; missing providers fail closed.

3.2 OPEN IMPLEMENTATION, INDEPENDENT DELIVERY DECISION

A complete disposable project. The solver operates on a project that includes the resources needed to understand and modify the game. This avoids reducing engine work to isolated snippets or text patches. Separate workspaces preserve the starting artifact, limit accidental cross-task state reuse, and make the delivered project available for evaluation. Workspace separation does not imply security isolation.

Model-led development. The solver receives the brief, workspace context, engine entry points, and execution constraints. It may compile, launch, inspect screenshots, debug, or write task-specific helpers during its session. Such actions belong to the solver trajectory. The harness records the session and enforces the session timeout and backend-specific limits, without prescribing a task-dependent sequence of tools or applying a repair after the final submission. Thus, “one session” can contain many edits and model-selected tests.

A separate evaluator. Once the solver exits, evaluation is carried out by an engine-native or benchmark-specific procedure. Its acceptance logic is not provided as an intermediate feedback channel in the solver protocol. Evaluation can use a separate copy of the artifact, preserving a distinction between delivered content and instrumentation. In native-open execution, however, this is a protocol-level distinction: broad host access does not itself prevent an adversarial solver from inspecting unrelated host files. Strong confidentiality requires an appropriately configured external isolation provider.

3.3 COMMON EXECUTION LAYER AND ENGINE ADAPTERS

The common execution layer separates the coding model, its permitted actions, and the target engine. A subscription-based coding session or an API-based interaction loop exchanges observations and actions with the model, while execution authority governs permitted operations. All experiments use subscription-based sessions; performance across model-access interfaces remains untested.

An engine adapter translates the common artifact contract into engine-specific operations: initializing the project context, selecting an engine version, and performing the required import, compilation, build, and runtime checks. This keeps project execution and assessment engine-specific while leaving the development policy to the solver.

Evaluation is specified as a set of checks appropriate to the task. Import and compilation assess basic engine compatibility; standalone builds and launches assess executable delivery; input replay and visual review assess observed behavior and requirement satisfaction. GameCraft-Bench combines build, replay, and rubric-based review, whereas the Unity creation suite uses independent Player builds and launch observations. Section 4 defines the checks used in each study. A pass is always interpreted relative to that study’s protocol, rather than as a guarantee of every property in Figure 1.

3.4 EVIDENCE AND FAILURE ATTRIBUTION

GameForge keeps the solver outcome distinct from engine and evaluator outcomes. A completed model session can still deliver an invalid project; a valid project can encounter a host launch failure;

Table 2: Recorded experimental conditions. The solver is `gpt-5.6-sol` throughout. Limits are per-task session timeouts, not matched realized compute. “Historical” denotes a separate local upstream run.

Suite	n	Engine	Effort / limit	Comparison
GameDevBench	332	Godot 4.4.1	Medium / 900 s	Historical local upstream, paired tasks
GameCraft-Bench	140	Godot 4.6.2	High / 7,200 s	Published aggregate reference
Godot Open20	20	Godot 4.4.1	Medium / 900 s	Historical local upstream, paired briefs
Unity Open20	20	Unity 6000.3	Medium / 1,200 s	Capability study; no comparator

and a replay can complete without satisfying a gameplay requirement. Collapsing these outcomes into the solver’s text report obscures which component failed.

We therefore interpret evidence in increasing levels of specificity. Import and compile gates establish basic engine compatibility. Standalone build and launch checks establish executable delivery under the tested configuration. Input replays establish the observed response to particular traces. Requirement-based visual review assesses recorded content under a rubric. Human review can add judgments about usability or progression, but it remains scoped to what was actually observed. These observations remain distinct in the final evaluation record.

4 EXPERIMENTAL SETUP

We study three questions: (1) can an open-session harness solve existing-project tasks competitively; (2) can the same design deliver full-game projects under build and replay evaluation; and (3) does the delivery contract extend across Godot and Unity? Table 2 summarizes the four suites. The suites were evaluated in separate experimental runs, and comparisons with historical conditions are identified explicitly.

Experimental design. All suites separate an open development session from final artifact evaluation. Each condition uses a fixed configuration, but engine access, prompting, and session persistence differ across the studies. Each scored task has one generation session and no automatic repair after final evaluation. The model may revise and test its project within that session, and process-level recovery may relaunch an engine command without regenerating the task.

GameDevBench. The upstream benchmark contains 333 tasks in the version used here (Chi et al., 2026). Before the full benchmark runs, we tested the supplied reference projects using the designated engine and official evaluator. One task, `task_0097`, was excluded because its reference project failed both an initial check and a repeat, owing to a required shader include that the evaluator did not accept. We retain the resulting 332-task denominator for both conditions; failures and timeouts do not reduce it. Both conditions use the same model, reasoning setting, engine, task order, four-worker concurrency, and 900-second timeout. “Local upstream” denotes the upstream Codex agent condition executed locally, not the benchmark’s public leaderboard row. The upstream and candidate runs completed on August 14 and August 17, 2026, respectively. The comparison evaluates the complete harness configuration, including its prompt and execution environment, rather than isolating the effect of a single component.

GameCraft-Bench. We use all 140 public tasks across 15 families (Luo et al., 2026). The collection includes 15 family-coverage pilot artifacts retained after configuration-compatibility checks and 125 additional generations. Generation used four parallel workers for most tasks; low scores did not trigger selective regeneration. The official Godot build, replay, frame selection, task rubrics, and task-specific aggregation formulas were retained. The benchmark’s default `gpt-5.5` visual judge ran through a Codex subscription transport at low reasoning, rather than the official API transport. Evaluation used the official Linux arm64 Godot build on Apple Silicon; the Kenney asset library was available and the OpenGameArt directory was empty. These differences require treating the quality score as a *subscription-judge approximation*, not a leaderboard-equivalent result.

Table 3: GameDevBench on the same 332 locally qualified tasks. The gain in correctness is accompanied by increased realized token use. Matched solver-only timing is unavailable for the retained candidate records.

Metric	GameForge	Local upstream	Difference
Tasks passed	213	184	+29
Pass rate	64.16%	55.42%	+8.73 pp
Tokens per task	422,736	276,739	+52.8%

Open-ended creation. The two Open20 suites use authored genre briefs, covering survival, platforming, puzzle, strategy, rhythm, racing, simulation, and cooperative mechanics. Each brief specifies a product goal without fixing source layout, scene names, APIs, or tool order. Godot starts from an empty project and compares the candidate with earlier local upstream artifacts. Both conditions allow native-engine execution; the candidate uses an ephemeral model session, while the upstream condition uses a non-ephemeral session and its standard prompt. These session and prompt differences are part of the system-level comparison. Each Open20 condition is executed serially. The Godot gate requires successful import and runtime probe, two nonempty screenshots, and metadata; it does not require a complete playthrough or a threshold of state change. A condition-blinded `gpt-5.6-sol` judge, at medium reasoning, sees the brief, implementation summary, runtime evidence, and images, while condition labels, trajectories, timing, and usage are withheld. Four dimensions—brief fulfillment, gameplay systems, visual coherence, and usability/feedback—are scored from 0 to 4. Unity uses engine version 6000.3.20f1, starts from a shared minimal project, and is evaluated by reimporting, compiling, building a macOS Player, launching it, and capturing two images. It has no upstream comparison arm.

Metrics and analysis. We report solver completion separately from artifact gates. GameDevBench uses the official task verdict; paired differences use the same task IDs. We compute a two-sided exact McNemar test on discordant outcomes and a percentile bootstrap that resamples 122 complete task families (20,000 samples). GameCraft-Bench reports the mean of official task-level reward formulas and category summaries; the overall reward is not reconstructed by averaging the four reported category means. Usage is measured from session telemetry, with cached input included within input tokens and reasoning output within output tokens. Subscription token counts are not converted to an API invoice. Statistical comparisons are exploratory, following system development.

5 RESULTS

5.1 EXISTING-PROJECT TASKS: HIGHER SUCCESS WITH MORE COMPUTE

GameForge solves 213/332 GameDevBench tasks, compared with 184/332 for the local upstream condition (Table 3). The absolute gain is 29 tasks, or 8.73 percentage points. Of the paired outcomes, 178 pass in both conditions, 35 pass only with GameForge, 6 only with local upstream, and 113 fail in both (Figure 2). The two-sided exact McNemar value is $p = 4.8736 \times 10^{-6}$. The task-family bootstrap gives a 95% interval of [5.65, 12.25] percentage points for the difference. These quantities indicate a consistent advantage in the observed pairing, but do not adjust for earlier harness iteration, selection of the frozen configuration, or unmeasured variation across repeated model runs.

All 332 candidate attempts received an ordinary official verdict: 213 passes and 119 failures, with no blocked/error outcomes or infrastructure retries recorded in the scored batch. Grouping the tasks by their publicly specified development requirements, candidate/local-upstream pass counts are 64/54 of 119 2D tasks, 8/6 of 9 3D tasks, 94/83 of 124 script/resource tasks, and 47/41 of 80 UI tasks. The nine-task 3D subset is too small to support broad claims about 3D development.

The success gain does not establish an efficiency gain. GameForge consumes 52.8% more tokens. Candidate timing includes final artifact evaluation, whereas upstream timing measures only the development session. Separate candidate session durations were not retained, so these measurements cannot support a matched solver-only timing comparison (Appendix B). Open exploration may allow a model to invest more work before declaring completion, but this experiment cannot distinguish

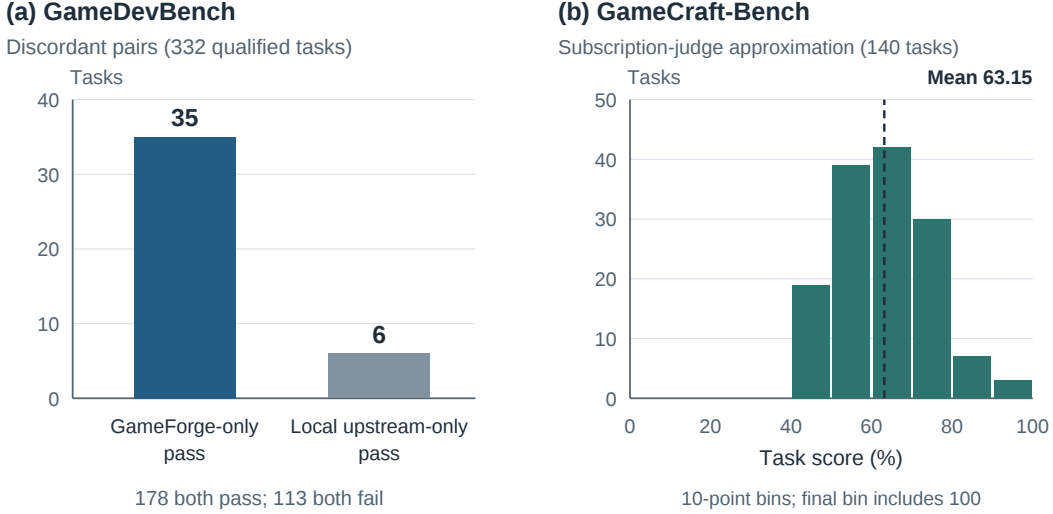


Figure 2: Complementary benchmark evidence. Left: discordant outcomes in the paired GameDevBench comparison; the net gain is 29 tasks. Right: the distribution of 140 GameCraft-Bench task scores under the disclosed subscription-judge protocol. Successful delivery of all projects does not imply uniformly high rubric satisfaction.

Table 4: GameCraft-Bench full140. The external Codex row is a published aggregate reference with the same solver label, not a rerun under matched conditions. Judge transport, evaluator architecture, and assets differ; the numerical gap is descriptive.

Metric	GameForge approx.	Published Codex	Gap (pp)
Overall	63.15%	60.50%	+2.65
Core Mechanics	76.15%	74.50%	+1.65
Content Depth	58.85%	56.10%	+2.75
Functional Visuals	68.74%	64.80%	+3.94
Presentation & Art	59.40%	57.00%	+2.40

the effect of that additional computation from other harness choices. An equal-compute ablation is therefore needed to attribute the improvement more narrowly. The public upstream result of 195/333 is useful context only: its denominator differs and public task-level pairing is unavailable.

5.2 FULL-GAME CREATION: RELIABLE DELIVERY, UNEVEN QUALITY

All 140 GameCraft-Bench solver sessions completed, all 140 official builds succeeded, and all 732 official interaction replays completed. No infrastructure failures or harness repair passes were recorded for the final scored attempts. The approximate mean score is 63.15%, with a median of 63.57%. Eighty-two tasks score at least 60%, 40 at least 70%, and 19 below 50%. Build and replay reliability therefore coexist with substantial remaining quality gaps.

Core mechanics receive the highest category mean (76.15%), while content depth and presentation/art remain near 59% (Table 4). This pattern suggests that producing a mechanically recognizable game is easier in this setting than delivering sustained content and polished presentation. It is a descriptive reading of the rubric, not a controlled decomposition of difficulty. The mean is 2.65 points above the published Codex row, but the unmatched conditions preclude a causal harness-superiority claim or a paired significance test. Rescoring the saved replay frames through the official API judge would resolve the judge-interface difference without regenerating the projects; a fully matched upstream rerun would still be needed for attribution.

The generation-and-import stage takes 1,392.08 minutes (9.94 minutes per task), including environment setup, project generation, and a final import check. The subsequent official build-and-replay

Table 5: Open-ended suites measure delivery under explicit gates. Scores are not comparable across engine suites, and hard-gate passes are not end-to-end playability certifications.

Condition	Completion	Gate pass	Scope of evidence
Godot / GameForge	20/20	20/20	Import, runtime probe, two captures
Godot / local upstream	20/20	20/20	Same gate; blinded quality comparison
Unity / GameForge	20/20	20/20	Import, compile, Player build and launch

stage takes 201.94 minutes, excluding visual judging. The run records 3,491 tool calls, 212.10 million input tokens (199.71 million cached), and 3.61 million output tokens. No comparable task-level resource record is available for the published reference, so these values describe the run rather than establish relative efficiency.

5.3 OPEN-ENDED CREATION ACROSS ENGINES

Both Godot conditions complete all 20 briefs and pass all 20 import/runtime-and-capture gates. The blinded mean quality totals are 15.15/16 for GameForge and 14.85/16 for local upstream, with a median paired difference of zero. Pairwise preference favors GameForge on 9 briefs and upstream on 11 (two-sided sign test $p = 0.8238$). This provides no detected preference advantage; it is not a formal equivalence result. In this comparison, GameForge uses 9.79% less elapsed time for development and environment setup, excluding final evaluation, and is faster on 13/20 briefs, but the small, historical, single-run comparison does not establish a stable cost improvement.

Unity Open20 achieves 20/20 solver completions and 20/20 independent import, compile, build, and Player-launch passes, with 40/40 valid 1280×720 captures. The solver independently produced standalone applications in 15 cases; the evaluator built all 20 projects regardless. The generation-and-compilation stage totals 7,822.4 seconds, with a median of 376.2 seconds; this includes final compilation and excludes the subsequent independent Player build and runtime evaluation. These results demonstrate transfer of the artifact-delivery contract to Unity. Without a comparison arm or task-specific full playthroughs, they do not quantify relative quality or complete gameplay correctness.

5.4 WHAT INTERACTION EVIDENCE ADDS

An additional interaction study examines eight Godot projects with engine-level input replay, one Unity project with operating-system mouse/keyboard replay, and four Unity projects using deterministic behavior tests implemented within the generated games. Figure 3 shows selected frames. These are curated evidence cases, not a random quality sample or an additional benchmark denominator. For example, the Unity deckbuilding replay documents a card reducing enemy health from 34 to 27, a shield increasing from 0 to 6, and a key event advancing to turn 2. Such state transitions support a stronger claim than a static launch image, while still falling short of a complete-game playthrough. Behavior tests initiated within a generated game provide runtime evidence, but do not independently verify that external input reaches the intended gameplay logic.

6 DISCUSSION AND LIMITATIONS

An interface contribution, not an isolated causal mechanism. The empirical results support the viability of open model-led development followed by independent artifact checks. They do not establish that any single design choice causes the observed GameDevBench gain. The compared systems differ in prompts, session behavior, and execution environment, and their realized compute differs despite matched maximum timeouts. Future ablations should hold these factors fixed and vary the project context, engine access, or final-evaluation interface separately.

Development exposure and generalization. These are retrospective evaluations after substantial harness iteration, using frozen configurations selected during development. The task sets were not untouched holdouts, and we did not repeat every condition over multiple model seeds. Paired tests and family bootstraps quantify uncertainty within the observed outcomes; they do not correct for

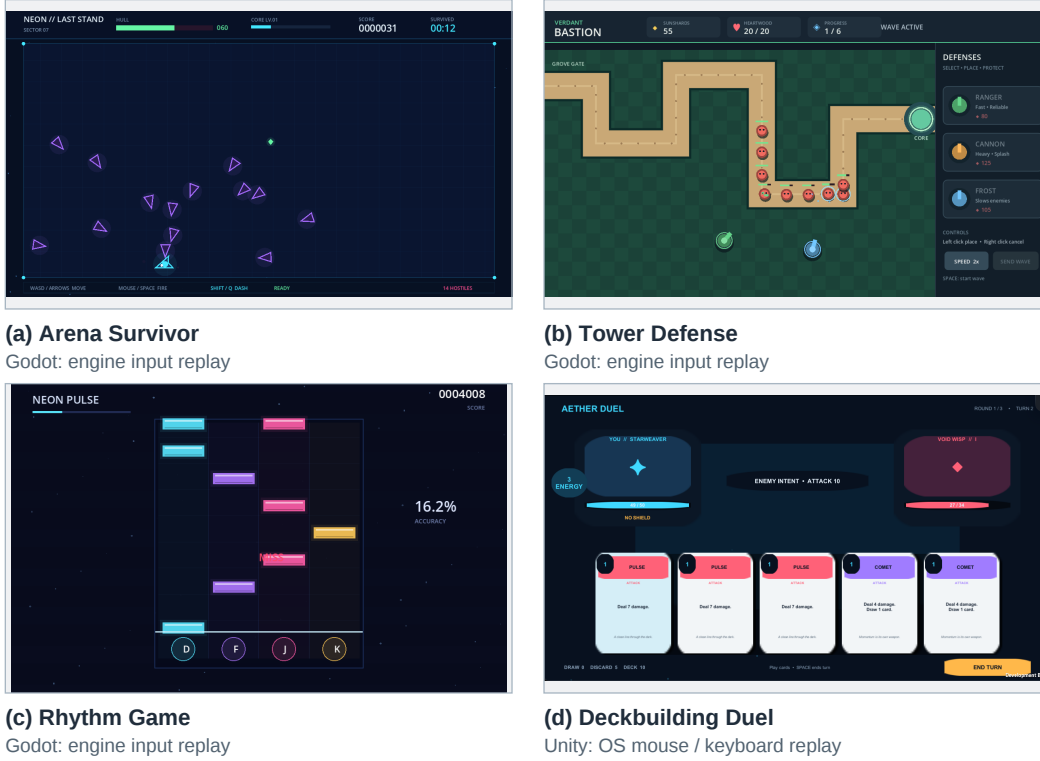


Figure 3: Selected unaltered frames from generated games. Godot cases use engine-level input replay; the Unity case uses operating-system mouse/keyboard events. These curated examples illustrate presentation and recorded interaction, not representative quality or complete playthrough coverage.

development selection or estimate model-run variance. The studies evaluate different stages of the system under their respective experimental conditions. Their results should therefore be interpreted within each task distribution and protocol, rather than pooled into a single estimate of general game-development ability. Repeated evaluations on previously unseen tasks would be needed to measure generalization more directly.

Verification coverage and judges. Gates cover only their implemented observations. Import and compilation checks cannot establish complete mechanics; a successful replay cannot establish responsiveness to untested inputs; and a two-frame probe cannot establish long-horizon consistency. Model-based judging can exhibit systematic biases (Zheng et al., 2023). The Godot quality judge belongs to the same model family as the solver, and GameCraft-Bench uses a disclosed transport approximation. Independent human playtesting, multiple judges, and held-out interaction traces would strengthen quality claims. The authored Open20 briefs deliberately cover genres and should not be interpreted as a sample from natural user demand.

Execution failures and authority. Open development includes errors: seven model-selected headless/movie-capture Godot invocations exited abnormally during Open20, after which the solver adapted within the same session. Successful final delivery does not imply an error-free trajectory. Recovery from an engine-process crash within a development session is distinct from regenerating a task after observing its final score. Native-open and supervised-native also do not provide strong protection against hostile generated code. External isolation remains provider-dependent, and host-side evaluation must be isolated separately when the threat model requires it.

7 FUTURE WORK

Unreal Engine. Extending the common artifact contract to Unreal Engine is a natural next step, but no Unreal capability is evaluated in this report. An adapter would need version-pinned project discovery, asset and plugin handling, C++ and Blueprint validation, packaging, and independent runtime interaction capture. Existing work on real C++ engine environments offers a relevant evaluation direction (La et al., 2026). The key test is whether these engine-specific responsibilities can be added while preserving the same open solver session and external delivery verdict.

Stronger comparisons and richer interaction. We plan matched upstream reruns with equal realized compute, repeated seeds, and a single evaluator environment. Longer design specifications and multi-session project evolution would test requirements persistence beyond short briefs, complementing specification-oriented game benchmarks (Lee et al., 2026). For creation quality, task-specific replay coverage, adversarial input sequences, human playtesting, and official API rescore of existing GameCraft-Bench frames would make the distance between “runs” and “satisfies the brief” more measurable. Separately testing model backends and authority profiles would distinguish interface portability from demonstrated performance portability.

8 CONCLUSION

GameForge organizes game development around a clear division of responsibility: the model decides how to build, while a separate evaluator determines what the delivered artifact demonstrates. Across the recorded studies, this design supports existing-project edits, complete-game creation, and runnable Unity/Godot delivery. The paired GameDevBench gain is substantial but compute-intensive; the creation studies show reliable engine delivery alongside remaining quality gaps. Treating projects, evaluation protocols, and evidence as explicit outputs provides a practical foundation for studying more capable game-development agents without conflating source-code generation, executable delivery, and verified gameplay.

REFERENCES

- Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Maria Elisabeth Bechtle, Feryal Behbahani, Stephanie C. Y. Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 4603–4623, 2024. URL <https://proceedings.mlr.press/v235/bruce24a.html>.
- Wayne Chi, Yixiong Fang, Arnav Yayavaram, Siddharth Yayavaram, Seth Karten, Qihong Anna Wei, Runkun Chen, Alexander Wang, Valerie Chen, Ameet Talwalkar, and Chris Donahue. GameDevBench: Evaluating agentic capabilities through game development. *arXiv preprint arXiv:2602.11103*, 2026. URL <https://arxiv.org/abs/2602.11103>. Version 2, June 30, 2026.
- Yilei Jiang, Jinyuan Hu, Qianyin Xiao, Yaozhi Zheng, Ruize Ma, Kaituo Feng, Jiaming Han, Tianshuo Peng, Kaixuan Fan, Manyuan Zhang, and Xiangyu Yue. OpenGame: Open agentic coding for games. *arXiv preprint arXiv:2604.18394*, 2026. URL <https://arxiv.org/abs/2604.18394>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Brian La, Sejoon Chang, Ben Kim, Junyoung Bae, Aamish Ahmad Beg, Sei Chang, Gonzalo Gonzalez-Pumariaga, and Kanav Goyal. GameEngineBench: Evaluating coding agents on real

- C++ runtime environments. *arXiv preprint arXiv:2607.03525*, 2026. URL <https://arxiv.org/abs/2607.03525>.
- Seonho Lee, Wonryeol Jeong, Alberto Cereser, Inha Kang, Hyeonjong Kim, Seungmin Kwak, and Dongmin Park. A2Z GameSpec-Bench: How faithfully can coding agents generate games from game design specifications? *arXiv preprint arXiv:2609.39564*, 2026. URL <https://arxiv.org/abs/2609.39564>.
- Tongxu Luo, Rongsheng Wang, Jiaxi Bi, Chenming Xu, Zhengyang Tang, Jianlong Chen, Juhao Liang, Ke Ji, Shuqi Guo, Yuhao Du, Fan Bu, Wenyu Du, Xiaotong Zhang, Kyle Li, Shaobo Wang, Linfeng Zhang, Yuxuan Liu, Xin Lai, Chenxin Li, Yiduo Guo, Zhixin Zhang, Xinyuan Wang, Tianyi Bai, Ziniu Li, and Benyou Wang. GameCraft-Bench: Can agents build playable games end-to-end in a real game engine? *arXiv preprint arXiv:2606.17861*, 2026. URL <https://arxiv.org/abs/2606.17861>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=P8pqeEkn1H>.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=OJd3ayDDoF>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. URL <https://arxiv.org/abs/2407.01489>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- Runxin Yang, Yuxuan Wan, Shuqing Li, and Michael R. Lyu. 90% faster, 100% code-free: MLLM-driven zero-code 3D game development. *arXiv preprint arXiv:2509.26161*, 2025. URL <https://arxiv.org/abs/2509.26161>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.05685>.

A STATISTICAL DETAILS

For GameDevBench, let $Y_i^G, Y_i^U \in \{0, 1\}$ denote the paired pass indicators. The empirical difference is

$$\hat{\Delta} = \frac{1}{332} \sum_{i=1}^{332} (Y_i^G - Y_i^U) = \frac{35 - 6}{332} = 0.0873494. \quad (3)$$

Conditioning on the 41 discordant tasks, the exact two-sided McNemar calculation is

$$p = 2 \sum_{j=0}^6 \binom{41}{j} 2^{-41} = 4.8735983 \times 10^{-6}. \quad (4)$$

The bootstrap resamples 122 task families with replacement, retaining all tasks in each selected family, and recomputes the task-weighted difference over 20,000 samples. The percentile interval is [0.0564516, 0.1225182]. This accounts for observed within-family dependence, but not development selection or model-run variation. The candidate’s Wilson 95% pass-rate interval is [58.86%, 69.13%].

For Godot Open20, the 9:11 preference split gives an exact two-sided sign-test value of 0.82380295. The non-significant preference result does not establish equivalence; neither the +0.30/16 mean score difference nor the zero median paired difference changes that interpretation.

B ADDITIONAL RESOURCE AND QUALITY DETAILS

Table 6: Recorded solver usage for GameForge. Cached input is a subset of input; it must not be added again. Output includes reasoning tokens. Counts are subscription telemetry, not billed API costs.

Suite	Input (cached) tokens	Output tokens	Tool calls
GameDevBench	138,668,935 (126,718,208)	1,679,298	4,355
GameCraft-Bench	212,100,397 (199,713,920)	3,613,849	3,491
Godot Open20	7,739,558 (6,936,960)	233,447	269
Unity Open20	16,557,775 (15,277,056)	311,094	349

Timing boundaries. The GameDevBench conditions use different timing boundaries: GameForge includes final evaluation, whereas the upstream measure covers only development. We therefore omit a solver-speed comparison. GameCraft-Bench’s 1,392.08 minutes include generation, import checks, and execution overhead; its subsequent build/replay stage takes 201.94 minutes. Unity’s 7,822.4 seconds cover generation and compilation, compared with 8,271.7 seconds including independent evaluation. Godot Open20 measures both conditions before final evaluation; its 9.79% time reduction includes environment setup and cleanup.

The Godot blinded review gives mean dimension scores (GameForge/upstream) of 4.00/3.80 for fulfillment, 3.95/3.65 for systems, 3.50/3.70 for visuals, and 3.70/3.70 for usability. The totals therefore obscure a small shift between implementation coverage and visual coherence. Unity’s separate non-blinded artifact review averages 15.1/16, but we do not use that descriptive score as comparative evidence.

C INTERACTION EVALUATION

The eight Godot cases cover arena survival, tower defense, rhythm, local co-op, stealth, match-three, fishing, and brick breaking. An external evaluator injects engine input events into unchanged generated projects. The Unity native-input case uses operating-system mouse and keyboard events; three events were delivered during a 10.6-second recording containing 106 frames. Four further Unity cases exercise deterministic behavior initiated within the games. These tests establish only the observed interactions. Figure 3 is illustrative, rather than representative of overall game quality.